



NEWSQL

Roberto Ibaceta Cisternas



ÍNDICE



01. NewSQL

02. Que no es NewSQL?

03. Los Pilares

04. Consenso y Tiempo

05. Cap Revisitado

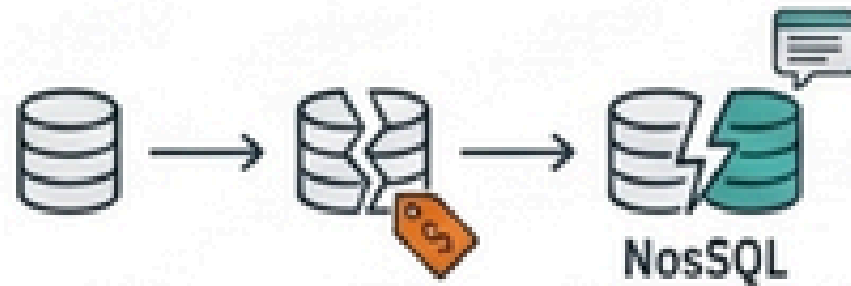
06. Ventajas

07. Los 3 Motores

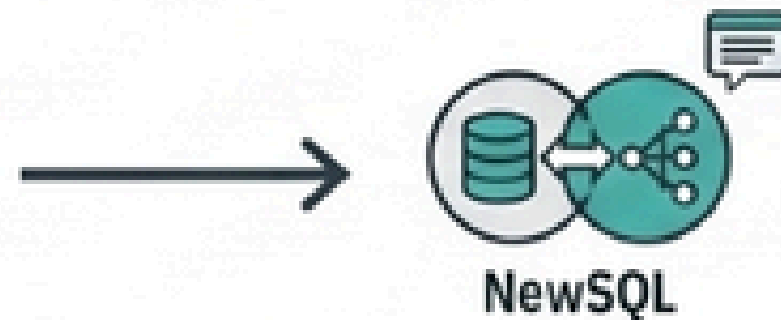
Hoja de Ruta



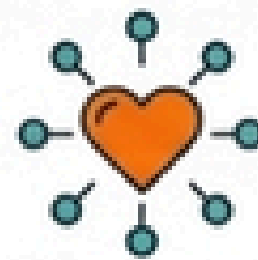
El Problema y la Promesa



De dónde venimos, qué nos costó NoSQL y la definición gráfica de NewSQL.



El Motor Bajo el Capó



Corazón Distribuido

Arquitectura Distribuida y Consenso

- Algoritmos de consenso (Raft/Paxos)
- Relojes atómicos (TrueTime)
- Manejo de Teorema PACELC

Realidad y Laboratorio



Los tres motores del mercado: Spanner (como principal inspiración), CockroachDB, VoltDB.



El Trato NoSQL



Escala horizontal infinita y tolerancia a fallas... a cambio de consistencia eventual (BASE).

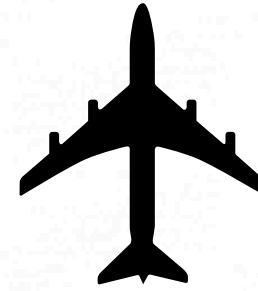
La respuesta de la industria fue obligar al programador a escribir código manual de reconciliación. Fue una solución frágil y carísima.

El Problema del Banco



¿Qué pasa con 200 millones de clientes? Los datos no caben en un servidor (necesitas NoSQL). Pero es dinero: no puedes permitir transferencias duplicadas ni saldos fantasmas (necesitas ACID).

¿Qué es NewSQL?

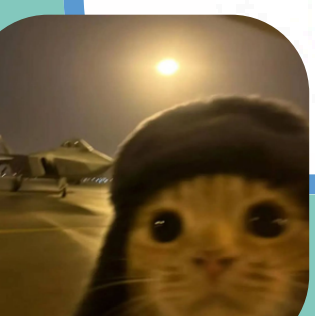


No es un SQL nuevo

El lenguaje es el mismo estándar de siempre. Lo que es completamente nuevo es el motor de base de datos rediseñado desde cero.

El 90% de Overhead

Un estudio demostró que los motores tradicionales gastaban el 90% de la CPU en bloqueos y logs, no en el trabajo real. El problema nunca fue ACID, fue cómo estaba implementado.



La Evolución



1970-2000

Reinado Relacional. Se escala verticalmente (comprando un servidor más grande). RDBMS domina.

 NoSQL

2011-Hoy

NewSQL. Nacen VoltDB, Spanner y CockroachDB. ACID y escalabilidad simultánea reescribiendo el motor.



2004-2009

Rompiendo el Techo. Google y Amazon superan los límites del servidor único. Nace Hadoop y la revolución NoSQL (Scale Out).

2008-2012

La Resaca BASE. Renunciar a ACID salió caro. Mantener el código de compensación en la aplicación supera el costo del hardware.



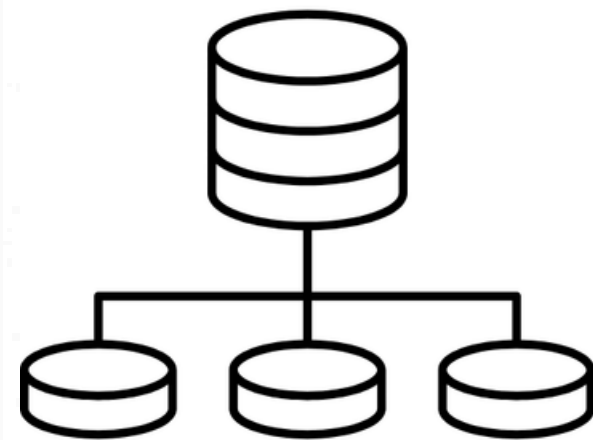
La Matriz de Convergencia



	SQL	NoSQL	NewSQL
Esquema:	Rígido	Flexible	Rígido
Escalamiento:	Vertical	Horizontal	Horizontal
Transacciones:	ACID local	BASE eventual	ACID distribuido
Lenguaje:	Estándar	APIs propias	SQL

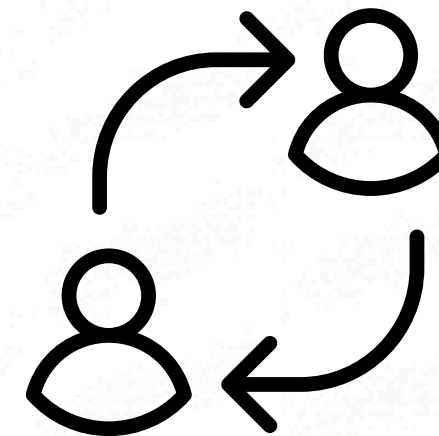
NewSQL se parece a NoSQL en una sola cosa: el escalamiento horizontal. En todo lo demás, es SQL clásico. (Migración de equipos altamente rentable).

Cómo lo logran: Los 4 Pilares



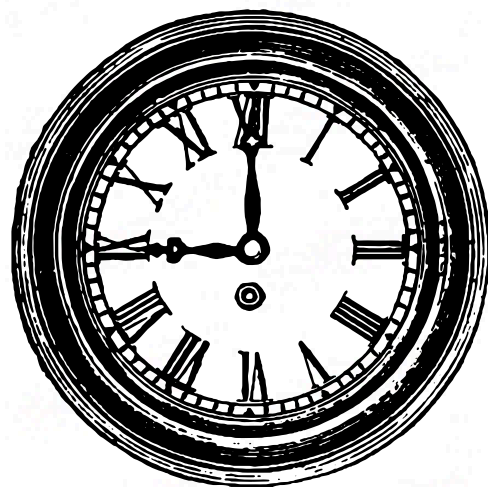
Particionamiento Automático (Sharding)

- Corte automático en trozos por rango de clave primaria.
- Rebalanceo dinámico.



Consenso Distribuido

- Replicación mediante algoritmos (Raft/Paxos).
- Reemplazo del esquema maestro-esclavo.



Relojes y Tiempo

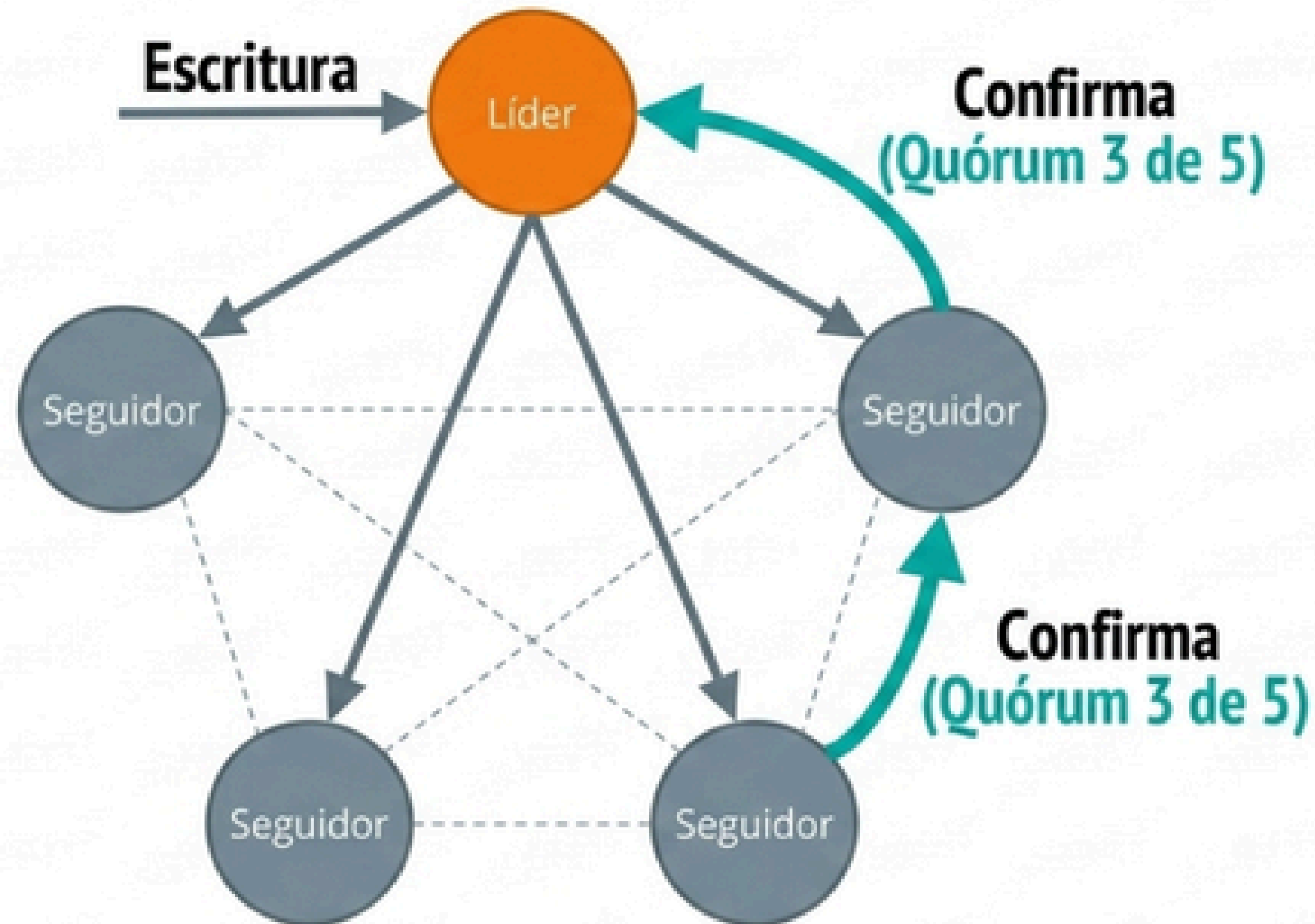
- Sincronización de eventos entre máquinas.
- Resolución del problema físico a nivel mundial.



Transacciones Abaratadas

- Uso de memoria RAM.
- Ejecución determinista sin bloqueos.
- Eliminación del overhead.

El Corazón Distribuido: Consenso (Raft)



Elección Aleatoria

Si el líder cae, los seguidores usan timeouts aleatorios para postularse. El que junta mayoría gana. Evita puntos únicos de falla.

Quórum de Mayoría

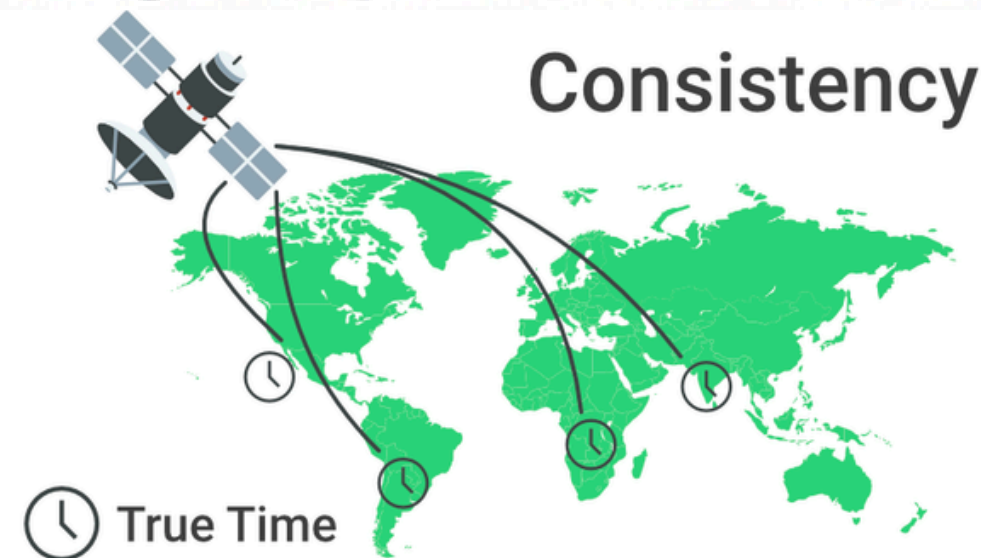
La escritura entra por el líder. El líder NO responde listo hasta que la mayoría de los nodos (ej. 3 de 5) confirme haber escrito.

Tolerancia Geométrica

Con 5 nodos, necesitas 3 firmas y toleras 2 caídos. Si la red se corta, la minoría se niega a escribir.

El Problema del Tiempo: ¿Qué pasó primero?

Google Spanner / TrueTime

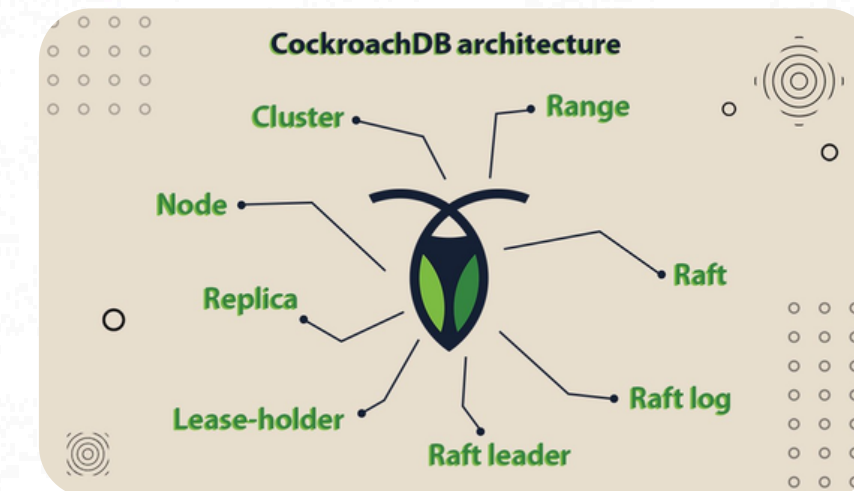


Solución de Hardware: Receptores GPS y relojes atómicos en cada datacenter.

La Magia: La API no devuelve una hora exacta, devuelve un intervalo de incertidumbre.

Commit-Wait: Spanner espera intencionalmente unos milisegundos a que pase la incertidumbre para garantizar un orden perfecto.

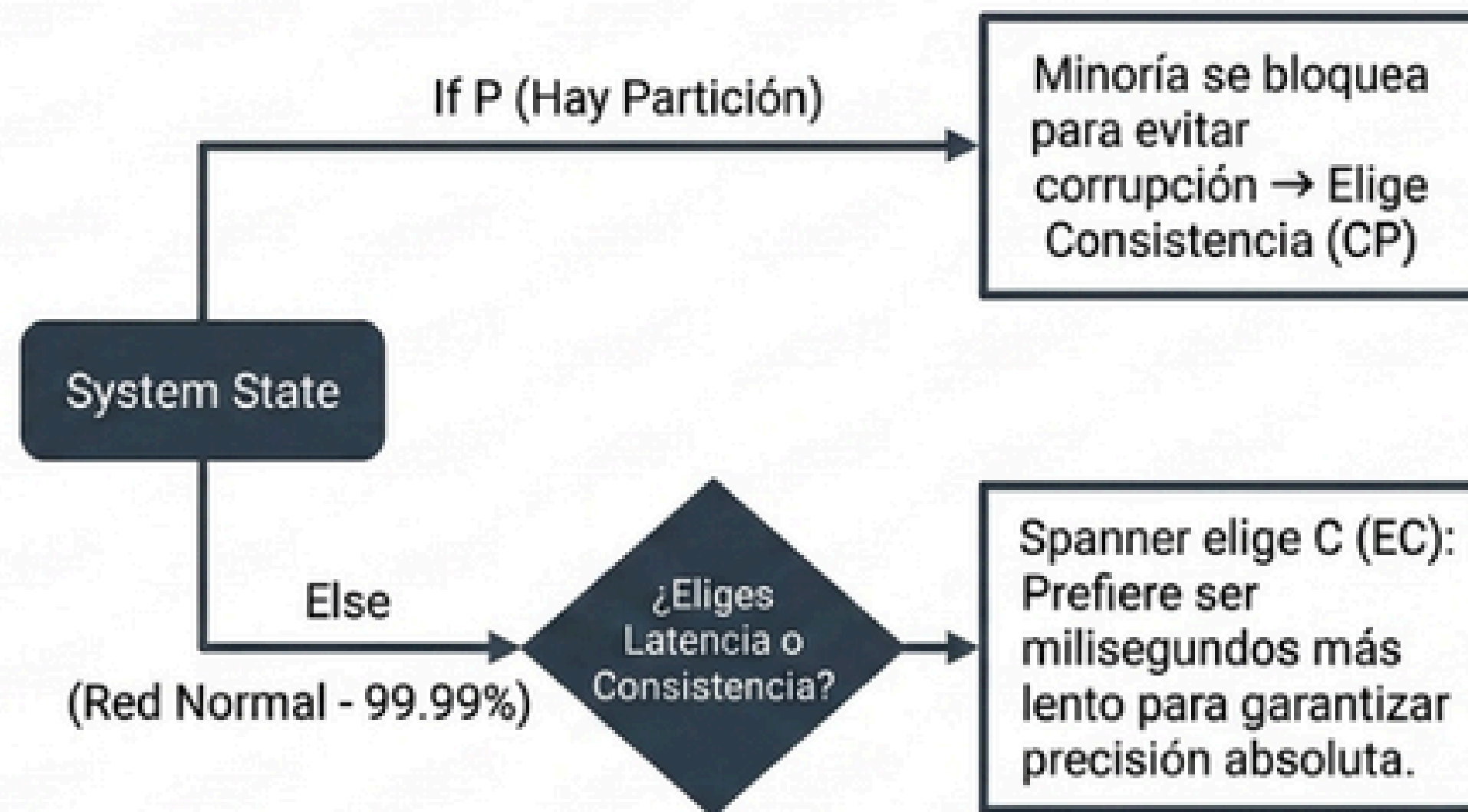
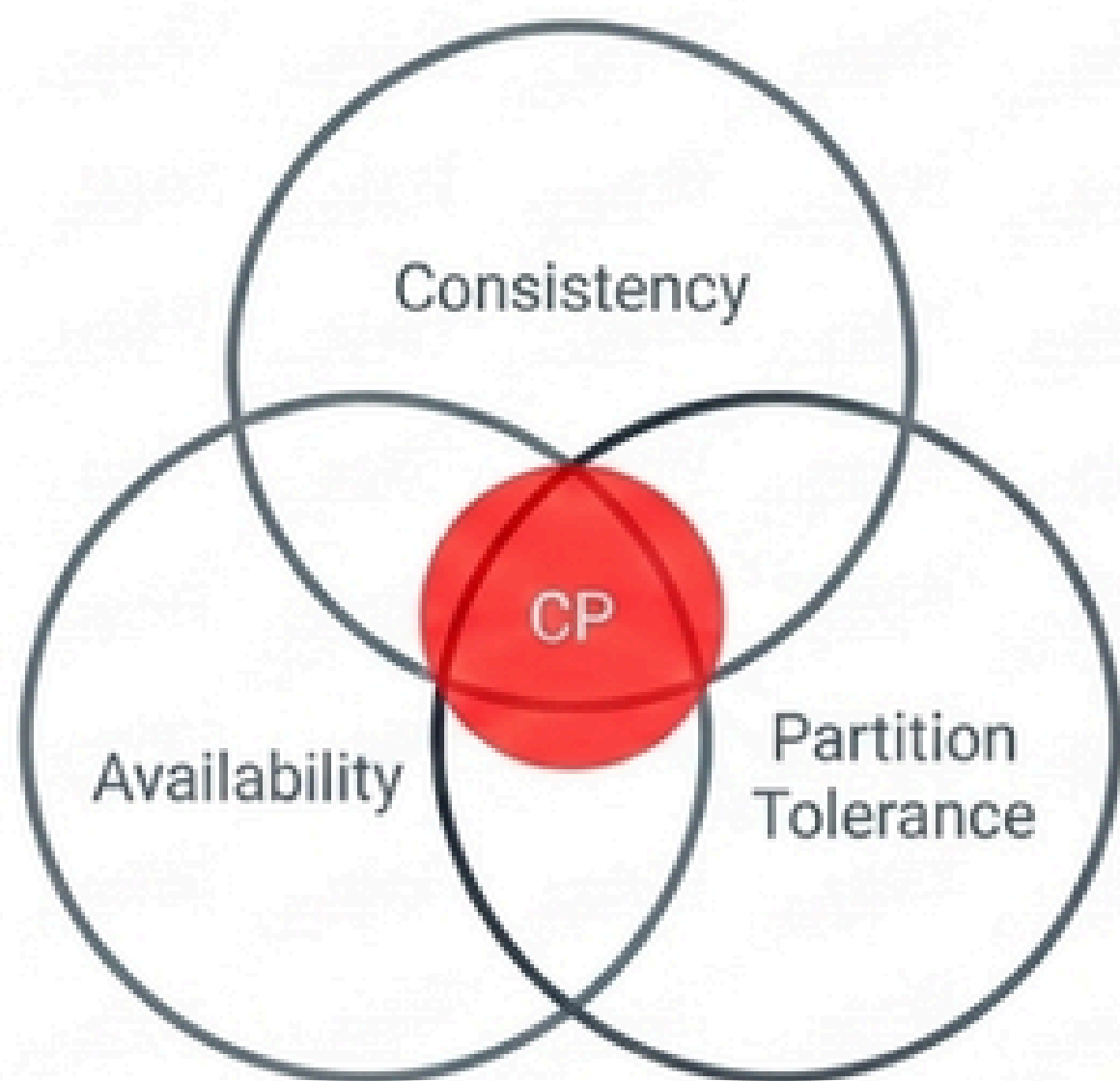
CockroachDB / HLC



Solución de Software: Relojes lógicos híbridos (hora física + contador lógico).

La Realidad: Si una lectura cae en zona de incertidumbre entre servidores, la transacción simplemente se aborta y se reintenta automáticamente.

El Teorema CAP Revisitado



" Spanner es efectivamente CA. - Eric Brewer. (La red privada de Google hace que las particiones sean tan raras que en la práctica ofrece 5 nueves de disponibilidad).

Ventajas Clave

- **Curva de Aprendizaje Cero:** Tu equipo ya sabe SQL. Migración barata.
- **Lógica Segura:** Las garantías ACID vuelven a la base, limpiando el código de la app.
- **Cero Mantenimiento:** Escala agregando nodos sin bajar el sistema.
- **Autosanación:** El consenso elige un líder nuevo automáticamente si un servidor muere.



Quién lo usa

- **Google:** Ads, Play, Gmail (Spanner).
- **Banca y Fintech:** Transacciones globales consistentes.
- **Retail y Telecom:** Inventario global, detección de fraude en milisegundos.



La Letra Chica: Desventajas y Cuándo NO Usarlo

Latencia Física

Nada viaja más rápido que la luz. Escribir requiere un viaje de ida y vuelta al quórum geográfico (decenas de milisegundos).

Costos Operacionales

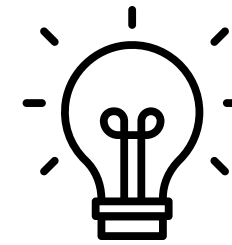
Un cluster base requiere múltiples nodos prendidos 24/7. Spanner es notoriamente caro.

Anti-patrón: ID Autoincremental

En NewSQL, IDs consecutivos envían todas las escrituras al mismo nodo, creando un cuello de botella (hotspot). Cambia las reglas de diseño.

Cuándo NO usarlo: Si tu carga es analítica (usa Data Warehouse), si no hay esquema definido (usa NoSQL), o si tus datos caben cómodamente en un solo servidor PostgreSQL bien indexado.

Los 3 Motores Principales

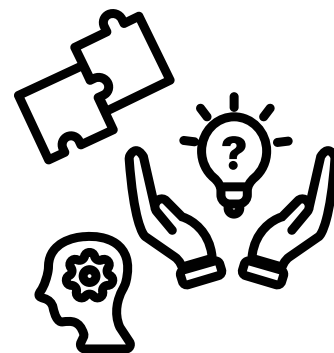


Google Spanner

Consenso: Paxos.

Tiempo: TrueTime.

Ideal para: Escala global absoluta (SaaS de Google).



CockroachDB

Consenso: Raft.

Tiempo: Relojes Híbridos.

Ideal para: Despliegue en cualquier nube, compatibilidad nativa con PostgreSQL.

VoltDB

Consenso: Replicación sincrónica.

Arquitectura: Todo en Memoria, hilo único por partición.

Ideal para: Fraude y telecomunicaciones.

3 Ideas para Llevarse

1

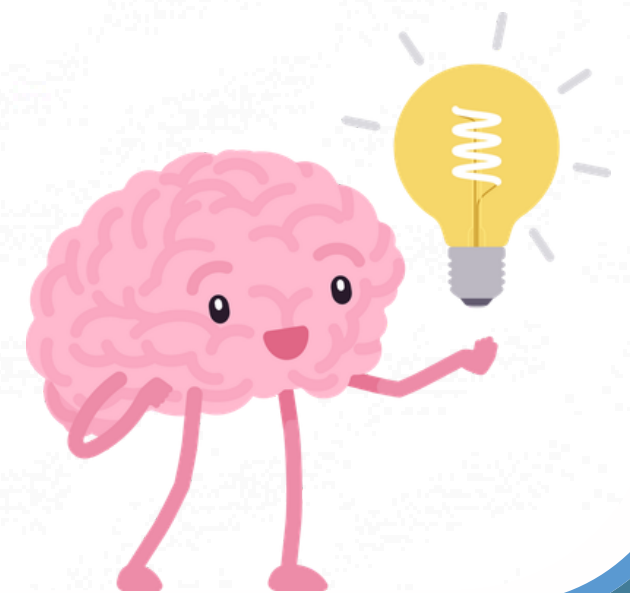
No es un SQL nuevo. Es el modelo relacional clásico operando sobre un **sistema distribuido masivo**.

2

El secreto de la infraestructura moderna es el **Quórum de Mayoría** (Raft). Tolerar caídas sin tener un punto único de falla central.

3

Diseñar sistemas es elegir qué te duele menos. Hoy, para recuperar el modelo ACID a escala global, pagamos con **latencia irreducible**.



CONTACTO

Gmail

- roberto.ibaceta@mail.udp.cl

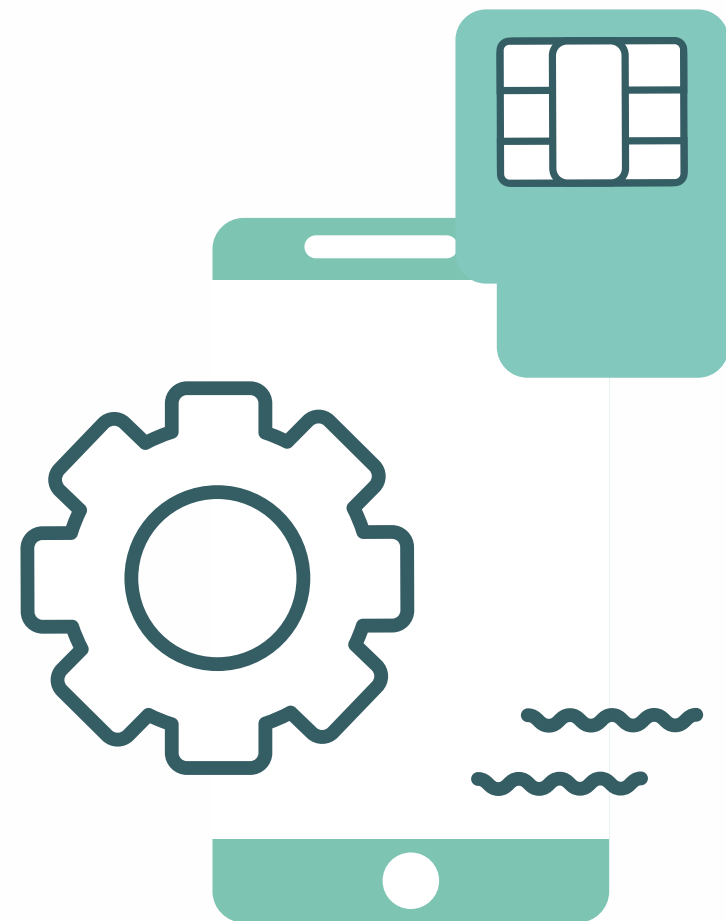
Wsp

- [+569 42663436](https://wa.me/56942663436)

Ds

- [roberibaz](#)





**MUCHAS
GRACIAS**

